

How to map words to space??

Informal algorithm for constructing word spaces

- Pick the words you are interested in: **target words**
- Define a **context window**, number of words surrounding target word
 - ▶ The context can in general be defined in terms of documents, paragraphs or sentences.
- Count number of times the target word co-occurs with the context words:
co-occurrence matrix
- Build vectors out of (a function of) these co-occurrence counts

An example:

- **Toy Corpus & Co-occurrence Matrix**

“data science drives insight”

“data analysis drives insight”

- Using a symmetrical **context window of size 1** (considering one word to the left and right), we construct a term-by-term co-occurrence matrix ***C***:

Target words ↓	data	science	analysis	drives	insight
data	0	1	1	0	0
science	1	0	0	1	0
analysis	1	0	0	1	0
drives	0	1	1	0	2
insight	0	0	0	2	2

An example: Raw co-occurrence counts:

- Computing Similarity: To quantify how close two word vectors $\mathbf{u}$ and $\mathbf{v}$ are in space, **Cosine Similarity** evaluates the angle between vectors rather than their magnitude:
- $$\text{Cosine Similarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{\sum_{i=1}^n u_i v_i}{\sqrt{\sum_{i=1}^n u_i^2} \sqrt{\sum_{i=1}^n v_i^2}}$$
- Using our matrix rows:
- $\mathbf{v}_{\text{science}} = [1, 0, 0, 1, 0]$
- $\mathbf{v}_{\text{analysis}} = [1, 0, 0, 1, 0]$
- $$\text{Cosine Similarity}(\mathbf{v}_{\text{science}}, \mathbf{v}_{\text{analysis}}) = \frac{(1 \cdot 1) + (0 \cdot 0) + (0 \cdot 0) + (1 \cdot 1) + (0 \cdot 0)}{\sqrt{1^2 + 1^2} \sqrt{1^2 + 1^2}} = \frac{2}{2} = 1.0$$
- Despite never co-occurring directly with each other, *science* and *analysis* map to identical directions in space because they share identical contexts (*data*, *drives*).
- Similarly find Cosine Similarity between all other pairs: $(\mathbf{v}_{\text{data}}, \mathbf{v}_{\text{analysis}})$; $(\mathbf{v}_{\text{drives}}, \mathbf{v}_{\text{analysis}})$; $(\mathbf{v}_{\text{insights}}, \mathbf{v}_{\text{analysis}})$; it will be different.

Issue with Raw co-occurrence counts:

- Raw co-occurrence counts suffer from frequency bias: common words (e.g., “*the*”, “*is*”) dominate matrix entries without providing informative semantic context.
- **Solution:** Context Weighting via association measures: Association measures adjust these weights.

Association measures are used to give more weight to contexts that are more significantly associated with a target word.

- The less frequent the target and context element are, the higher the weight given to their co-occurrence count should be.
- different measures - e.g., Mutual information, Log-likelihood ratio

Pointwise Mutual Information (PMI)

- PMI quantifies whether two words co-occur significantly more often than expected by random chance:
- $$\text{PMI}(w, c) = \log_2 \left(\frac{P(w, c)}{P(w)P(c)} \right) = \log_2 \left(\frac{C(w, c) \cdot N}{C(w) \cdot C(c)} \right)$$
- Where N is the total sum of co-occurrence counts across the matrix, $C(w)$ is the marginal count of word w , and $C(w, c)$ is their joint co-occurrence count.
- **Positive PMI (PPMI):** Because $\text{PMI} \rightarrow -\infty$ as $P(w, c) \rightarrow 0$, practical NLP applications replace negative values with zero:
- $$\text{PPMI}(w, c) = \max(0, \text{PMI}(w, c))$$

Statistically measure whether two words co-occur frequently (relative to their global frequencies)

Apply PPMI to Toy Corpus

- **Total co-occurrences (N):** $2 + 2 + 2 + 4 + 2 = 12$
- **PMI Calculations**
- $\text{PMI}(w, c) = \log_2 \left(\frac{C(w, c) \cdot N}{C(w) \cdot C(c)} \right)$, $\text{PPMI}(w, c) = \max(0, \text{PMI}(w, c))$
- (data, science)**and** (data, analysis):
- $\text{PMI} = \log_2 \left(\frac{1 \times 12}{2 \times 2} \right) = \log_2(3) \approx \mathbf{1.585}$
- (science, drives)**and** (analysis, drives):
- $\text{PMI} = \log_2 \left(\frac{1 \times 12}{2 \times 4} \right) = \log_2(1.5) \approx \mathbf{0.585}$
- (drives, insight):
- $\text{PMI} = \log_2 \left(\frac{2 \times 12}{4 \times 2} \right) = \log_2 \left(\frac{24}{8} \right) = \log_2(3) \approx \mathbf{1.585}$
- **Non-co-occurring pairs ($C(w, c) = 0$):** $\text{PMI} \rightarrow -\infty \Rightarrow \text{PPMI} = \mathbf{0}$

Apply PPMI to Toy Corpus

Final PPMI Matrix

	data	science	analysis	drives	insight
data	0	1.585	1.585	0	0
science	1.585	0	0	0.585	0
analysis	1.585	0	0	0.585	0
drives	0	0.585	0.585	0	1.585
insight	0	0	0	1.585	0

Key Outcomes

- **Semantic Synonymy:** $\mathbf{v}_{\text{science}} = [1.585, 0, 0, 0.585, 0]$ and $\mathbf{v}_{\text{analysis}} = [1.585, 0, 0, 0.585, 0]$ are identical vectors (Cosine Similarity = 1.0) because they share identical statistical contexts.
- **Frequency Adjustment:** Although "*drives*" co-occurs twice with "*insight*" ($C = 2$) and only once with "*science*" ($C = 1$), the higher overall frequency of "*drives*" ($C = 4$) adjusts the associations proportionally (1.585 vs. 0.585).

Simpler Method: TF-IDF (Term Frequency – Inverse Document Frequency)

- Use documents as context
- While PMI measures word-word association, TF-IDF weights words relative to document contexts in a **term-document matrix**:
- $TF\text{-}IDF(t, d, D) = TF(t, d) \times IDF(t, D)$
 - $TF(t, d)$: Frequency of term t in document d .
 - D represents the entire corpus (the collection of all documents) in dataset.
 - IDF Down-weights words appearing across all documents while elevating rare, highly discriminative keywords.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

$$IDF(t, D) = \log\left(\frac{\text{Total Number of documents in corpus } D}{\text{Number of documents containing term } t}\right)$$

TF-IDF is a statistical measure used to evaluate the importance of a word in a document within a collection or corpus of documents.

Example of TF-IDF

Document 1: "The cat sat on the mat."

Document 2: "The dog played in the park."

Document 3: "Cats and dogs are great pets. "

Total number of documents in the corpus(D) =3

Calculation of TF score for the word "cat"

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

	No of times cat appears	Total number of terms	TF score
Document 1	1	6	1/6
Document 2	0	6	0/6
Document 3	1	6	1/6

Note: Applied Lemmatization to words: Cat and Dog

Example of TF-IDF

Calculation of IDF score for the word “cat”

Total number of documents in the corpus (D):	3	$IDF(cat, D)$
Number of documents containing the term "cat":	2 (Document 1 and Document 3).	$\log \frac{3}{2} \approx 0.176$

$IDF(t, D) = \log\left(\frac{\text{Total Number of documents in corpus } D}{\text{Number of documents containing term } t}\right)$

$TF-IDF(t, d, D) = TF(t, d) * IDF(t, D)$

For Document 1: $TF-IDF(cat, Document 1, D) = 0.167 * 0.176 = 0.029$

For Document 2: $TF-IDF(cat, Document 2, D) = 0 * 0.176 = 0$

For Document 3: $TF-IDF(cat, Document 3, D) = 0.167 * 0.176 = 0.029$

- **Problem:** $\log(\infty) \rightarrow$ approaches ∞ and $\log(1) = 0 \rightarrow$ whenever a term appears in every document: IDF is always 0.
- **Solution (Sklearn TF-IDF vectorizer):** Modified $IDF(t, D) = \ln\left(\frac{1 + \text{Total Number of Documents in corpus } D}{1 + \text{Number of documents containing term } t}\right) + 1$
- Use L_2 Normalization for every document after finding TF-IDF for every score

A Python Example: TF – IDF Vectorizer

```
sentences = [  
'The stock price of google jumps on the earning data today',  
'Google plunge on China Data!'  
]  
from sklearn.feature_extraction.text import TfidfVectorizer  
vectorizer = TfidfVectorizer(stop_words='english')  
TFIDF = vectorizer.fit_transform(sentences)  
print(vectorizer.get_feature_names_out())  
print(TFIDF.shape)  
print(TFIDF.toarray())
```

- Output
['china' 'data' 'earning' 'google' 'jumps' 'plunge' 'price' 'stock'
'today']
(2, 9)
[[0. 0.29017021 0.4078241 0.29017021 0.4078241 0.
0.4078241 0.4078241 0.4078241]
[0.57615236 0.40993715 0. 0.40993715 0. 0.57615236
0. 0. 0.]]

Note: The default tokenizer lowercases text, removes punctuation, and filters standard English stop words (the, of, on):

Are Python results with theoretical calculations?

- **Document 1 (Sentence 1):**
- **Unnormalized vector:**
 - data, google: $1 \times 1.0000 = 1.0000$
 - earning, jumps, price, stock, today: $1 \times 1.4055 = 1.4055$
- **L2 Norm:**
- $\|v_1\|_2 = \sqrt{2 \times (1.0)^2 + 5 \times (1.4055)^2} \approx \sqrt{11.8767} \approx 3.4463$
- **Normalized scores:**
 - data, google: $\frac{1.0}{3.4463} \approx \mathbf{0.2902}$
 - earning, jumps, price, stock, today: $\frac{1.4055}{3.4463} \approx \mathbf{0.4078}$
 - china, plunge: $\mathbf{0.0000}$
- **Document 2 (Sentence 2):**
- **Unnormalized vector:**
 - data, google: $1 \times 1.0000 = 1.0000$
 - china, plunge: $1 \times 1.4055 = 1.4055$
- **L2 Norm:**
- $\|v_2\|_2 = \sqrt{2 \times (1.0)^2 + 2 \times (1.4055)^2} \approx \sqrt{5.9507} \approx 2.4394$
- **Normalized scores:**
 - china, plunge: $\frac{1.4055}{2.4394} \approx \mathbf{0.5762}$
 - data, google: $\frac{1.0}{2.4394} \approx \mathbf{0.4099}$
 - All other terms: $\mathbf{0.0000}$